Your VPCs (1) Info

Last updated 12 minutes ago | Actions | Create VPC

Find VPCs by attribute or tag

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	I
monitoring-vpc	vpc-05eed01960ebd4fcc	Available	–	–	Off	10.0.0.0/16	–

Subnets (1) Info

Last updated 13 minutes ago | Actions | Create subnet

Find subnets by attribute or tag

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDI
public-subnet	subnet-069ee102b5e4e6580	Available	vpc-05eed01960ebd4fcc | moni...	Off	10.0.1.0/24	–

Internet gateways (1) Info

Actions | Create internet gateway

Find internet gateways by attribute or tag

Name	Internet gateway ID	State	VPC ID	Owner
monitoring-IGW	igw-0196b1973ba624e5c	Attached	vpc-05eed01960ebd4fcc | monitoring-vpc	451637428602

Route tables (1) Info

Last updated less than a minute ago | Actions | Create route table

Find route tables by attribute or tag

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC	Owner ID
cloudwatch-automation-RT	rtb-0db26e438d2fe8c57	subnet-069ee102b5e4e6...	–	Yes	vpc-05eed01960ebd4fcc | moni...	451637428602

Security Groups (1/2) Info

Actions | Export security groups to CSV | Create security group

Find security groups by attribute or tag

Name	Security group ID	Security group name	VPC ID	Description
–	sg-0602f99dcef96e4be	monitoring-SG	vpc-05eed01960ebd4fcc	monitoring-SG

Instances (1) Info

Connect | Instance state | Actions | Launch instances

Find Instance by attribute or tag (case-sensitive) | All states

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public
monitoring-se...	i-0957207b56c717f12	Running	t3.micro	3/3 checks passec	View alarms +	ap-south-1a	–	13.23!

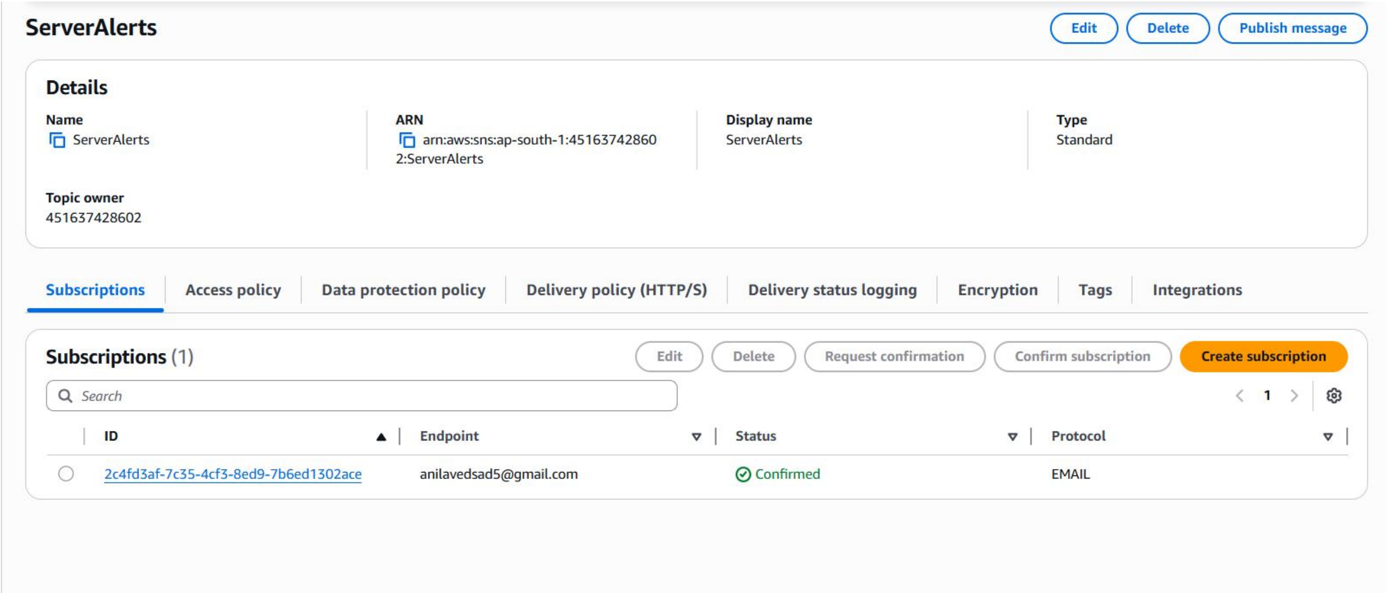
Roles (7) Info

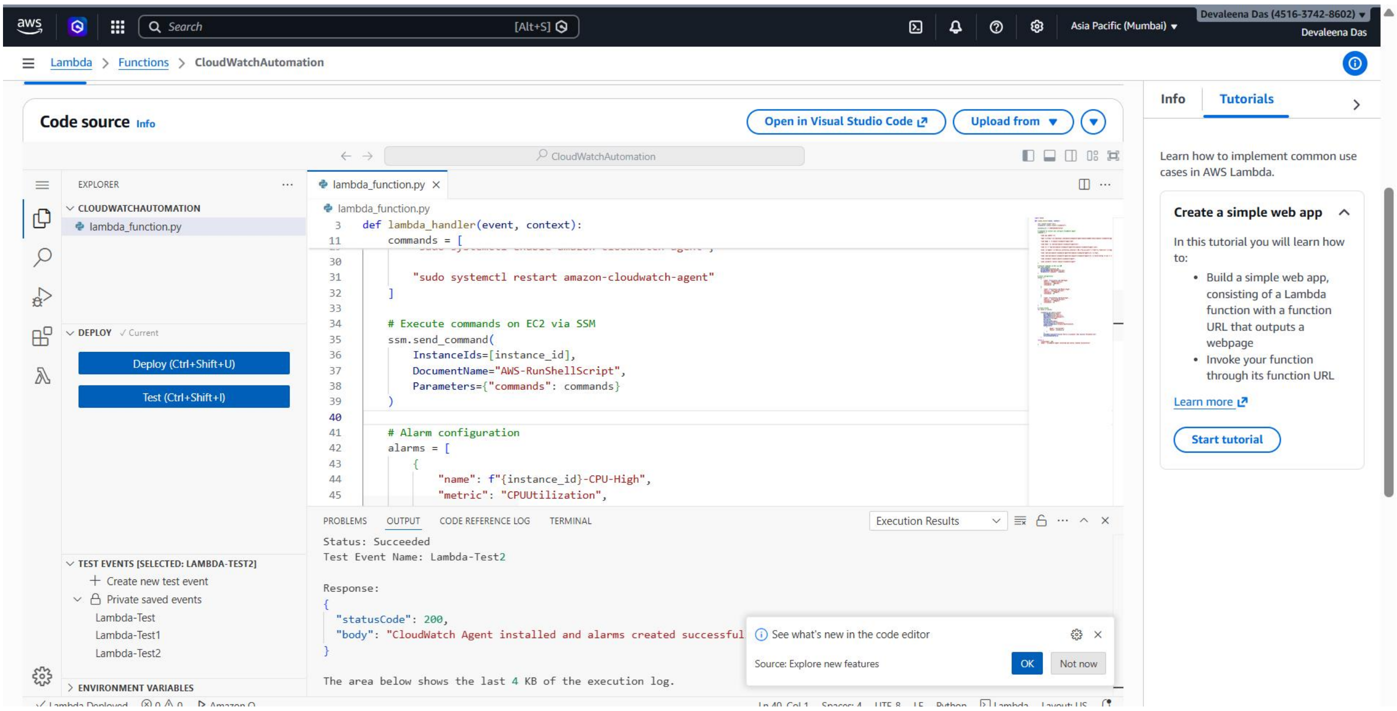
Delete | Cre

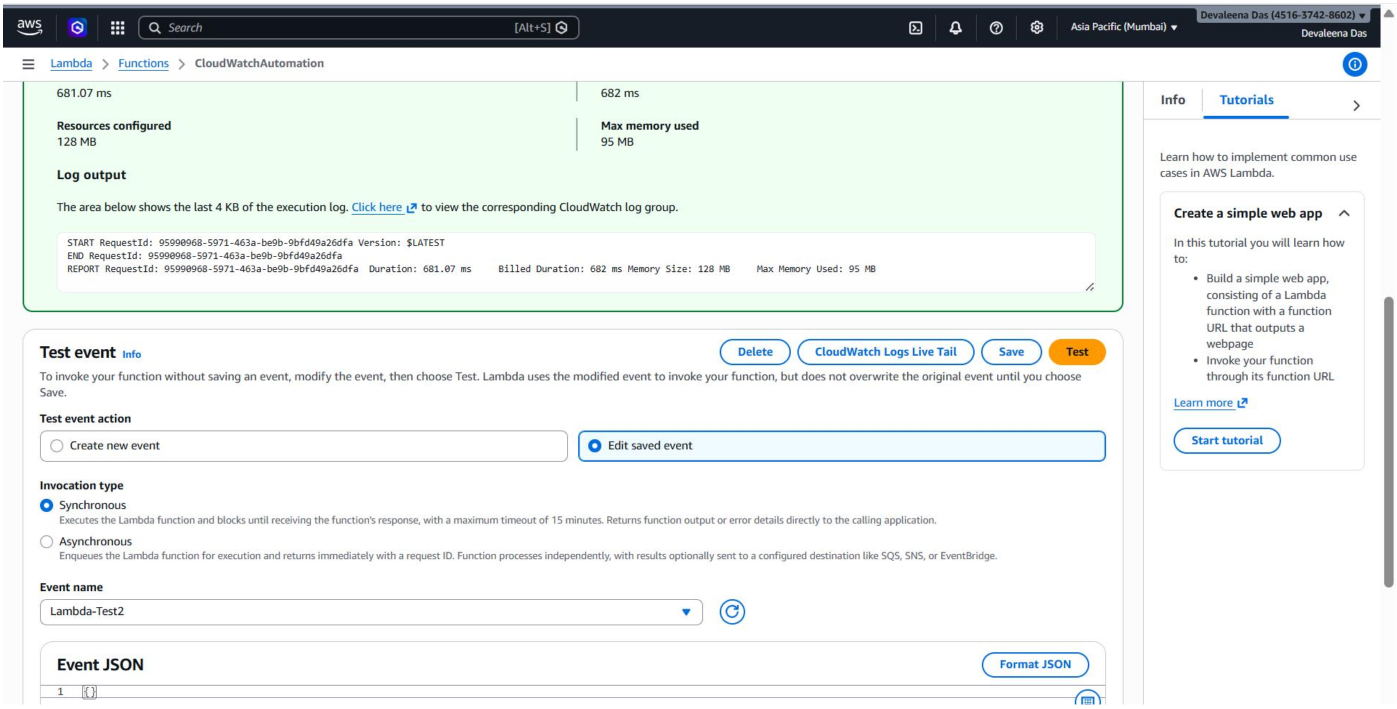
An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

Role name	Trusted entities	Last activity
AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Se	7 minutes ago
AWSServiceRoleForSupport	AWS Service: support (Service-Linkec	-
AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service	-
CloudWatchAutomation	AWS Service: lambda	-
CloudWatchAutomation-role-gjipvt85	AWS Service: lambda	1 hour ago
EC2-CloudWatch-Role	AWS Service: ec2	8 minutes ago
ec2-ssm-role	AWS Service: ec2	-


ServerAlerts
Edit
Delete
Publish message
Details
Name
ServerAlerts
ARN
arn:aws:sns:ap-south-1:45163742860
2:ServerAlerts
Display name
ServerAlerts
Type
Standard
Topic owner
451637428602
Subscriptions
Access policy
Data protection policy
Delivery policy (HTTP/S)
Delivery status logging
Encryption
Tags
Integrations
Subscriptions (1)
Edit
Delete
Request confirmation
Confirm subscription
Create subscription
Search
ID
Endpoint
Status
Protocol
2c4fd3af-7c35-4cf3-8ed9-7b6ed1302ace
anilavedsad5@gmail.com
Confirmed
EMAIL


Search
[Alt+S]
Asia Pacific (Mumbai)
Devaleena Das (4516-3742-8602)
Devaleena Das
Lambda
Functions
CloudWatchAutomation
Code source
Info
Open in Visual Studio Code
Upload from
CloudWatchAutomation
EXPLORER
CLOUDWATCHAUTOMATION
lambda_function.py
DEPLOY
Current
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
TEST EVENTS [SELECTED: LAMBDA-TEST2]
Create new test event
Private saved events
Lambda-Test
Lambda-Test1
Lambda-Test2
ENVIRONMENT VARIABLES
def lambda_handler(event, context):
commands = [
"sudo systemctl restart amazon-cloudwatch-agent"
]
Execute commands on EC2 via SSM
ssm.send_command(
InstanceIds=[instance_id],
DocumentName="AWS-RunShellScript",
Parameters={"commands": commands}
)
# Alarm configuration
alarms = [
{
"name": f"{instance_id}-CPU-High",
"metric": "CPUUtilization",
PROBLEMS
OUTPUT
CODE REFERENCE LOG
TERMINAL
Execution Results
Status: Succeeded
Test Event Name: Lambda-Test2
Response:
{
"statusCode": 200,
"body": "CloudWatch Agent installed and alarms created successful
}
The area below shows the last 4 KB of the execution log.
See what's new in the code editor
Source: Explore new features
OK
Not now
Info
Tutorials
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial


Search
[Alt+S]
Asia Pacific (Mumbai)
Devaleena Das (4516-3742-8602)
Devaleena Das
Lambda
Functions
CloudWatchAutomation
681.07 ms
682 ms
Resources configured
128 MB
Max memory used
95 MB
Log output
The area below shows the last 4 KB of the execution log. Click here to view the corresponding CloudWatch log group.
START RequestId: 95990968-5971-463a-be9b-9bfd49a26dfa Version: $LATEST
END RequestId: 95990968-5971-463a-be9b-9bfd49a26dfa
REPORT RequestId: 95990968-5971-463a-be9b-9bfd49a26dfa Duration: 681.07 ms Billed Duration: 682 ms Memory Size: 128 MB Max Memory Used: 95 MB
Test event Info
Delete
CloudWatch Logs Live Tail
Save
Test
To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.
Test event action
Create new event
Edit saved event
Invocation type
Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.
Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.
Event name
Lambda-Test2
Event JSON
Format JSON
1 {}
Info
Tutorials
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial

Code:

```python
import boto3

def lambda_handler(event, context):

    ssm = boto3.client('ssm')
    cloudwatch = boto3.client('cloudwatch')

    instance_id = "i-0957207b56c717f12"

    # Commands to install and configure CloudWatch Agent
    commands = [

        "sudo apt update -y",

        "wget -q https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb",

        "sudo dpkg -i -E amazon-cloudwatch-agent.deb",

        "sudo mkdir -p /opt/aws/amazon-cloudwatch-agent/etc",

        "sudo rm -f /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json",

        "echo '{\"agent\":{\"metrics_collection_interval\":60,\"run_as_user\":\"root\"},\"metrics\":{\"append_dimensions\":{\"InstanceId\":\"${aws:InstanceId}\"},\"metrics_collected\":{\"mem\":{\"measurement\":[\"mem_used_percent\"],\"metrics_collection_interval\":60},\"disk\":{\"resources\":[\"/\"],\"measurement\":[\"used_percent\"],\"metrics_collection_interval\":60}}}}' | sudo tee /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json",

        "sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a stop",

        "sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -s",

        "sudo systemctl enable amazon-cloudwatch-agent",

        "sudo systemctl restart amazon-cloudwatch-agent"
    ]

    # Execute commands on EC2 via SSM
    ssm.send_command(
        InstanceIds=[instance_id],
        DocumentName="AWS-RunShellScript",
        Parameters={"commands": commands}
    )

    # Alarm configuration
    alarms = [
        {
            "name": f"{instance_id}-CPU-High",
            "metric": "CPUUtilization",
            "namespace": "AWS/EC2",
            "threshold": 80
        },
        {
            "name": f"{instance_id}-Memory-High",
            "metric": "mem_used_percent",
            "namespace": "CWAgent",
            "threshold": 80
        },
        {
            "name": f"{instance_id}-Disk-High",
            "metric": "disk_used_percent",
            "namespace": "CWAgent",
            "threshold": 80
        }
    ]

    # Create alarms
    for alarm in alarms:
```

```python
    cloudwatch.put_metric_alarm(
        AlarmName=alarm["name"],
        MetricName=alarm["metric"],
        Namespace=alarm["namespace"],
        Statistic="Average",
        Period=300,
        EvaluationPeriods=1,
        Threshold=alarm["threshold"],
        ComparisonOperator="GreaterThanThreshold",
        Dimensions=[
            {
                "Name": "InstanceId",
                "Value": instance_id
            }
        ],
        AlarmDescription=f"{alarm['metric']} greater than {alarm['threshold']}%",
        ActionsEnabled=False
    )

return {
    "statusCode": 200,
    "body": "CloudWatch Agent installed and alarms created successfully"
}
```